

BAB 4: KOMPONEN UI + EVENT HANDLING

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa diharapkan mampu:

1. Mengidentifikasi dan memahami fungsi komponen UI dasar Android.
2. Membuat layout menggunakan XML untuk menampilkan berbagai komponen UI.
3. Menangani event pengguna (seperti klik, pilihan, dll) pada komponen UI menggunakan Kotlin.
4. Menerapkan konsep event listener untuk membuat aplikasi yang interaktif.

4.1 Pengenalan Komponen UI Dasar

Antarmuka pengguna (User Interface/UI) pada Android dibangun menggunakan hierarki objek **View** dan **ViewGroup**. **View** adalah objek yang mewakili elemen UI di layar, seperti tombol, teks, atau gambar. **ViewGroup** adalah wadah tak terlihat yang mendefinisikan struktur layout untuk View dan ViewGroup lainnya, seperti `LinearLayout`, `RelativeLayout`, atau `ConstraintLayout`.

Berikut adalah beberapa komponen UI dasar yang paling sering digunakan:

- `TextView` : Menampilkan teks statis ke pengguna.
- `EditText` : Memungkinkan pengguna untuk memasukkan teks. Ini adalah subclass dari `TextView`.
- `Button` : Tombol yang dapat ditekan oleh pengguna untuk menjalankan aksi.
- `ImageView` : Menampilkan gambar dari sumber daya (resource) aplikasi atau dari URL.
- `CheckBox` : Kotak centang yang dapat dicentang atau tidak dicentang oleh pengguna. Ideal untuk pilihan yang bisa lebih dari satu.
- `RadioButton` : Tombol radio yang biasanya digunakan dalam sebuah `RadioGroup`. Hanya satu `RadioButton` dalam satu `RadioGroup` yang dapat dipilih.
- `Spinner` : Komponen dropdown yang memungkinkan pengguna memilih satu item dari daftar.

4.2 Konsep Event Handling

Aplikasi yang interaktif merespons aksi dari pengguna. Aksi ini disebut **event**. Contoh event adalah klik tombol, perubahan teks pada `EditText`, atau pemilihan item pada `Spinner`. Untuk menangani event, kita menggunakan mekanisme yang disebut **Event Listener**.

Event Listener adalah sebuah antarmuka (interface) yang menunggu event tertentu terjadi pada komponen UI. Ketika event terjadi, metode (callback) dalam listener tersebut akan dieksekusi.

Beberapa jenis listener yang umum:

- `OnClickListener` : Menangani event klik pada View (Button, TextView, dll).
- `OnCheckedChangeListener` : Menangani perubahan status pada CheckBox atau RadioButton.
- `OnItemSelectedListener` : Menangani pemilihan item pada Spinner.

4.3 Implementasi Praktik: Aplikasi Sapaan

Kita akan membuat aplikasi sederhana yang meminta nama pengguna melalui `EditText`, dan ketika sebuah `Button` ditekan, aplikasi akan menampilkan sapaan di `TextView`. Kita juga akan menambahkan `CheckBox` untuk memberikan pilihan sapaan formal/informal.

Langkah 1: Membuat Layout (`activity_main.xml`)

Buka file `res/layout/activity_main.xml` dan tambahkan komponen-komponen berikut. Kita akan menggunakan `ConstraintLayout` untuk mengatur posisi komponen.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/tvLabel"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Masukkan Nama Anda:"
        android:textSize="18sp"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <EditText
        android:id="@+id/etName"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:hint="Contoh: John Doe"
        android:inputType="textPersonName"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/tvLabel" />

    <CheckBox
        android:id="@+id/cbFormal"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```

```

        android:layout_marginTop="16dp"
        android:text="Gunakan sapaan formal?"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/etName" />

<Button
    android:id="@+id/btnSapa"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="24dp"
    android:text="Sapa!"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/cbFormal" />

<TextView
    android:id="@+id/tvResult"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="32dp"
    android:textSize="20sp"
    android:textStyle="bold"
    tools:text="Halo, John Doe!"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/btnSapa" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

Langkah 2: Menulis Logika di Kotlin (MainActivity.kt)

Sekarang, kita akan menambahkan logika untuk menangani event di MainActivity.kt .

```

package com.example.sapaanapp // sama dengan package nama app

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button
import android.widget.CheckBox
import android.widget.EditText
import android.widget.TextView
import android.widget.Toast

```

```

class MainActivity : AppCompatActivity() {
    // Deklarasi variabel untuk komponen UI
    private lateinit var etName: EditText
    private lateinit var cbFormal: CheckBox
    private lateinit var btnSapa: Button
    private lateinit var tvResult: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // Inisialisasi variabel dengan komponen dari layout
        etName = findViewById(R.id.etName)
        cbFormal = findViewById(R.id.cbFormal)
        btnSapa = findViewById(R.id.btnSapa)
        tvResult = findViewById(R.id.tvResult)

        // Menangani event klik pada Button
        btnSapa.setOnClickListener {
            // Ambil teks dari EditText
            val name = etName.text.toString()

            // Periksa apakah nama kosong
            if (name.isEmpty()) {
                // Tampilkan pesan singkat jika nama kosong
                Toast.makeText(this, "Nama tidak boleh kosong!",
                    Toast.LENGTH_SHORT).show()
            } else {
                // Tentukan sapaan berdasarkan status CheckBox
                val greeting = if (cbFormal.isChecked) {
                    "Selamat pagi, Bapak/Ibu $name"
                } else {
                    "Halo, $name!"
                }
                // Tampilkan sapaan di TextView
                tvResult.text = greeting
            }
        }

        // Menangani event perubahan pada CheckBox (opsional, sebagai contoh)
        cbFormal.setOnCheckedChangeListener { buttonView, isChecked ->
            val message = if (isChecked) "Mode sapaan formal diaktifkan" else
                "Mode sapaan informal diaktifkan"
            Toast.makeText(this, message, Toast.LENGTH_SHORT).show()
        }
    }
}

```

```
}  
  
}
```

4.4 Latihan Praktikum

Studi Kasus: Kalkulator Sederhana

Buatlah aplikasi kalkulator sederhana yang dapat melakukan operasi penambahan dua bilangan.

Ketentuan:

1. Gunakan dua `EditText` untuk input bilangan pertama dan kedua. Set `inputType`-nya menjadi `numberDecimal`.
2. Gunakan tiga `RadioButton` untuk memilih operasi (+, -, *).
3. Gunakan satu `Button` untuk menghitung hasil.
4. Tampilkan hasil perhitungan pada sebuah `TextView`.
5. Jika salah satu input kosong, tampilkan `Toast` dengan pesan "Input tidak boleh kosong!".

BAB 5: FORM INPUT & VALIDASI

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa diharapkan mampu:

1. Mendesain dan membangun form input yang kompleks dengan berbagai komponen UI.
2. Memahami pentingnya validasi input dalam aplikasi.
3. Mengimplementasikan berbagai jenis validasi (wajib isi, format, panjang karakter).
4. Memberikan feedback visual yang jelas kepada pengguna saat input tidak valid menggunakan `setError()`.

5.1 Konsep Form Input dan Validasi

Form Input adalah kumpulan komponen UI yang dirancang untuk mengumpulkan data dari pengguna. Contohnya adalah form registrasi, form login, atau form biodata. Dalam pengembangan aplikasi, form adalah elemen krusial untuk interaksi data.

Validasi Input adalah proses memeriksa data yang dimasukkan pengguna untuk memastikan data tersebut sesuai dengan aturan yang telah ditentukan sebelum data diproses lebih lanjut (misalnya, dikirim ke server atau disimpan di database).

Mengapa Validasi Penting?

- **Integritas Data:** Memastikan data yang tersimpan atau dikirim adalah benar dan konsisten.
- **Keamanan:** Mencegah input berbahaya (seperti SQL injection atau XSS).
- **User Experience (UX):** Memberikan panduan yang jelas kepada pengguna tentang bagaimana mengisi form dengan benar, mengurangi frustrasi.

5.2 Metode Validasi di Android

Ada beberapa cara untuk memberikan feedback hasil validasi:

1. **Toast** : Pesan singkat yang muncul sebentar di layar. Cocok untuk notifikasi umum.
2. **Snackbar** : Mirip Toast, tetapi dapat ditampilkan di bagian bawah layar dan bisa memiliki aksi (seperti tombol "Tutup").
3. **setError()** pada **EditText** / **TextInputLayout** : Cara yang paling direkomendasikan untuk validasi form. Metode ini akan menampilkan pesan error tepat di bawah komponen input yang bermasalah, memberikan konteks yang sangat jelas kepada pengguna. Penggunaan **TextInputLayout** dari Material Design library lebih disarankan karena memberikan tampilan error yang lebih baik dan fitur floating label.

5.3 Implementasi Praktik: Form Registrasi

Kita akan membuat form registrasi dengan validasi untuk nama, email, password, dan konfirmasi password. Kita akan menggunakan **TextInputLayout** untuk mencapai hasil terbaik.

Langkah 1: Tambahkan Dependensi Material Design

Pastikan file **build.gradle.kts** (Module :app) Anda sudah memiliki dependensi berikut:

```
dependencies {
    // ... dependensi lainnya
    implementation("com.google.android.material:material:1.12.0") // Gunakan versi
    terbaru
}
```

Dan pastikan tema aplikasi Anda di **res/values/themes.xml** mewarisi dari tema Material Components, misalnya **Theme.Material3.DayNight**.

Langkah 2: Membuat Layout (**activity_registration.xml**)

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp"
    tools:context=".RegistrationActivity">
```

```
<TextView
    android:id="@+id/tvTitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Buat Akun Baru"
    android:textSize="24sp"
    android:textStyle="bold"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/tilNama"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginTop="24dp"
    android:hint="Nama Lengkap"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/tvTitle">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/etNama"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textPersonName" />
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/tilEmail"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:hint="Email"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/tilNama">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/etEmail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textEmailAddress" />
</com.google.android.material.textfield.TextInputLayout>
```

```

<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/tilPassword"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:hint="Password"
    app:passwordToggleEnabled="true"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/tilEmail">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/etPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textPassword" />
</com.google.android.material.textfield.TextInputLayout>

<Button
    android:id="@+id/btnDaftar"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginTop="32dp"
    android:text="DAFTAR"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/tilPassword" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

Langkah 3: Menulis Logika Validasi di Kotlin (`RegistrationActivity.kt`)

```

package com.example.registration // ganti dengan package name anda

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button
import android.widget.Toast
import com.google.android.material.textfield.TextInputEditText
import com.google.android.material.textfield.TextInputLayout

class RegistrationActivity : AppCompatActivity() {

```

```

private lateinit var tilNama: TextInputLayout
private lateinit var tilEmail: TextInputLayout
private lateinit var tilPassword: TextInputLayout
private lateinit var etNama: TextInputEditText
private lateinit var etEmail: TextInputEditText
private lateinit var etPassword: TextInputEditText
private lateinit var btnDaftar: Button

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_registration)

    // Inisialisasi View
    tilNama = findViewById(R.id.tilNama)
    tilEmail = findViewById(R.id.tilEmail)
    tilPassword = findViewById(R.id.tilPassword)
    etNama = findViewById(R.id.etNama)
    etEmail = findViewById(R.id.etEmail)
    etPassword = findViewById(R.id.etPassword)
    btnDaftar = findViewById(R.id.btnDaftar)

    btnDaftar.setOnClickListener {
        if (validateForm()) {
            // Jika semua validasi berhasil
            Toast.makeText(this, "Registrasi Berhasil!",
                Toast.LENGTH_SHORT).show()
            // Di sini biasanya akan ada proses lanjutan, seperti mengirim
            data ke server
        }
    }
}

private fun validateForm(): Boolean {
    val name = etNama.text.toString().trim()
    val email = etEmail.text.toString().trim()
    val password = etPassword.text.toString().trim()

    // Reset error
    tilNama.error = null
    tilEmail.error = null
    tilPassword.error = null

    var isValid = true

    // Validasi Nama

```

```
        if (name.isEmpty()) {
            tilNama.error = "Nama tidak boleh kosong"
            isValid = false
        }

        // Validasi Email
        if (email.isEmpty()) {
            tilEmail.error = "Email tidak boleh kosong"
            isValid = false
        } else if (!android.util.Patterns.EMAIL_ADDRESS.matcher(email).matches())
        {
            tilEmail.error = "Format email tidak valid"
            isValid = false
        }

        // Validasi Password
        if (password.isEmpty()) {
            tilPassword.error = "Password tidak boleh kosong"
            isValid = false
        } else if (password.length < 8) {
            tilPassword.error = "Password minimal 8 karakter"
            isValid = false
        }

        return isValid
    }
}
```

5.4 Studi Kasus Latihan Praktikum

Studi Kasus: Form Login dengan Validasi

Buatlah sebuah Activity baru untuk form login dengan ketentuan berikut:

1. **Input:** Email dan Password.
2. **Validasi:**
 - Email tidak boleh kosong dan harus format email yang valid.
 - Password tidak boleh kosong.
3. **Logika:**
 - Jika email = "admin@example.com" dan password = "password123", tampilkan **Toast** "Login Berhasil!".
 - Jika salah, tampilkan **Toast** "Email atau Password salah!".
4. Gunakan **TextInputLayout** untuk menampilkan pesan error.

BAB 6: ACTIVITY LIFECYCLE + INTENT

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa diharapkan mampu:

1. Memahami siklus hidup (lifecycle) dari sebuah Activity dan setiap tahapannya.
2. Menjelaskan kapan setiap metode callback lifecycle dipanggil.
3. Memahami konsep Intent untuk navigasi antar Activity.
4. Mengimplementasikan Intent Explicit untuk berpindah Activity dan mengirim data.
5. Mengimplementasikan Intent Implicit untuk memanfaatkan aplikasi lain di perangkat.

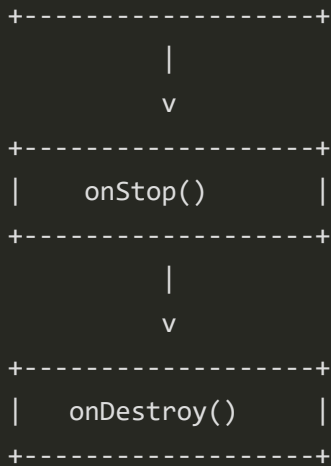
6.1 Konsep Activity Lifecycle

Activity adalah salah satu komponen fundamental dalam aplikasi Android yang menyajikan satu layar dengan antarmuka pengguna. Selama hidupnya, sebuah Activity akan melalui beberapa keadaan (states). Transisi antar keadaan ini diatur oleh **Activity Lifecycle**, yang dikelola oleh sistem operasi Android.

Memahami lifecycle sangat penting untuk mengelola sumber daya dengan baik dan mencegah crash (misalnya, mencoba mengakses komponen yang sudah tidak ada).

Diagram Activity Lifecycle:





Penjelasan Metode Callback:

- **onCreate()** : Dipanggil saat Activity pertama kali dibuat. Di sinilah kita melakukan inisialisasi satu kali, seperti **setContentView()**, membuat daftar data, dan menghubungkan variabel dengan View di layout.
- **onStart()** : Dipanggil ketika Activity akan terlihat oleh pengguna, tetapi belum interaktif.
- **onResume()** : Dipanggil ketika Activity berada di latar depan (foreground) dan siap menerima input dari pengguna. Activity berada di state "Running".
- **onPause()** : Dipanggil ketika Activity akan digantikan oleh Activity lain (misalnya dialog muncul atau Activity lain masuk ke latar depan). Di sini kita harus menyimpan data atau state yang belum disimpan, dan menghentikan animasi atau proses yang memakan CPU.
- **onStop()** : Dipanggil ketika Activity tidak lagi terlihat oleh pengguna. Ini terjadi jika pengguna beralih ke Activity lain atau menekan tombol home.
- **onDestroy()** : Dipanggil sebelum Activity dihancurkan oleh sistem. Ini adalah panggilan terakhir yang diterima Activity. Gunakan untuk membersihkan semua sumber daya yang belum dibersihkan.
- **onRestart()** : Dipanggil sebelum **onStart()** ketika Activity yang sudah dihentikan (**onStop()**) akan ditampilkan kembali kepada pengguna.

6.2 Konsep Intent

Intent adalah objek pesan yang dapat digunakan untuk meminta aksi dari komponen aplikasi lain. Meskipun Intent memfasilitasi komunikasi antar komponen dengan berbagai cara, ada tiga kasus penggunaan utamanya:

1. **Memulai sebuah Activity:** Untuk memulai layar baru.
2. **Memulai sebuah Service:** Untuk memulai operasi di latar belakang.
3. **Mengirimkan Broadcast:** Untuk mengkomunikasikan bahwa suatu peristiwa telah terjadi.

Ada dua jenis Intent:

- **Intent Explicit:** Menentukan komponen target secara spesifik (misalnya, `com.example.DetailActivity`). Digunakan untuk komunikasi internal dalam aplikasi Anda sendiri.

- **Intent Implicit:** Tidak menentukan komponen target, tetapi mendeskripsikan aksi umum yang akan dilakukan (misalnya, "buka halaman web" atau "tampilkan peta"). Sistem Android akan menemukan aplikasi yang dapat menangani aksi tersebut.

6.3 Implementasi Praktik: Mengirim Data Antar Activity

Kita akan membuat dua Activity:

1. **FormActivity** : Berisi form untuk memasukkan nama dan umur.
2. **ResultActivity** : Menampilkan data yang dikirim dari **FormActivity** .

Langkah 1: Buat Layout **FormActivity** (**activity_form.xml**)

```
<!-- activity_form.xml -->
<LinearLayout ... android:orientation="vertical" ...>
    <TextView ... android:text="Form Biodata" ... />
    <EditText android:id="@+id/etNama" ... android:hint="Nama" />
    <EditText android:id="@+id/etUmur" ... android:hint="Umur"
    android:inputType="number" />
    <Button android:id="@+id/btnKirim" ... android:text="Kirim Data" />
</LinearLayout>
```

Langkah 2: Buat Layout **ResultActivity** (**activity_result.xml**)

```
<!-- activity_result.xml -->
<LinearLayout ... android:orientation="vertical" ...>
    <TextView android:id="@+id/tvResult" ... android:text="Data Diterima"
    android:textSize="18sp"/>
</LinearLayout>
```

Langkah 3: Buat Activity Baru

Klik kanan pada package -> New -> Activity -> Empty Views Activity, beri nama **ResultActivity** .

Langkah 4: Logika di **FormActivity.kt** (Mengirim data)

```
class FormActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
```

```

    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_form)

    val etNama = findViewById<EditText>(R.id.etNama)
    val etUmur = findViewById<EditText>(R.id.etUmur)
    val btnKirim = findViewById<Button>(R.id.btnKirim)

    btnKirim.setOnClickListener {
        val nama = etNama.text.toString()
        val umur = etUmur.text.toString()

        // Membuat Intent Explicit
        val intent = Intent(this, ResultActivity::class.java)

        // Menambahkan data ke Intent
        intent.putExtra("EXTRA_NAMA", nama)
        intent.putExtra("EXTRA_UMUR", umur)

        // Memulai Activity baru
        startActivity(intent)
    }
}

```

Langkah 5: Logika di `ResultActivity.kt` (Menerima data)

```

class ResultActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_result)

        val tvResult = findViewById<TextView>(R.id.tvResult)

        // Menerima data dari Intent
        val nama = intent.getStringExtra("EXTRA_NAMA")
        val umur = intent.getStringExtra("EXTRA_UMUR")

        // Menampilkan data
        val text = "Nama: $nama\nUmur: $umur tahun"
        tvResult.text = text
    }
}

```

Langkah 6: Contoh Intent Implicit

Tambahkan tombol di `FormActivity` untuk membuka URL.

```
// ...  
  
val btnOpenWeb = findViewById<Button>(R.id.btnOpenWeb) // Asumsi tombol ini ada di XML  
  
btnOpenWeb.setOnClickListener {  
    val webpage = Uri.parse("https://www.google.com")  
    val intent = Intent(Intent.ACTION_VIEW, webpage)  
    if (intent.resolveActivity(packageManager) != null) {  
        startActivity(intent)  
    } else {  
        Toast.makeText(this, "Tidak ada aplikasi yang dapat membuka web",  
            Toast.LENGTH_SHORT).show()  
    }  
}
```

6.4 Studi Kasus Latihan Praktikum

Studi Kasus: Aplikasi Data Mahasiswa

1. Buat sebuah aplikasi dengan dua Activity: `InputMhsActivity` dan `ProfileMhsActivity`.
2. Di `InputMhsActivity`, buat form dengan input: NIM, Nama, dan Jurusan (gunakan `Spinner` untuk pilihan: Teknik Informatika, Sistem Informasi, Manajemen).
3. Saat tombol "Lihat Profile" ditekan, kirim semua data tersebut ke `ProfileMhsActivity`.
4. Di `ProfileMhsActivity`, tampilkan semua data yang diterima dengan format yang rapi.
5. Tambahkan juga sebuah tombol di `ProfileMhsActivity` yang menggunakan Intent Implicit untuk memanggil aplikasi telepon dengan nomor yang sudah ditentukan (misalnya nomor universitas).

BAB 7: RECYCLERVIEW UNTUK DATA LIST

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa diharapkan mampu:

1. Memahami konsep dan keunggulan `RecyclerView` dibandingkan `ListView`.
2. Menjelaskan peran dari `Adapter`, `ViewHolder`, dan `LayoutManager`.
3. Menerapkan `RecyclerView` untuk menampilkan sekumpulan data dalam bentuk daftar (list).
4. Membuat aplikasi daftar kontak sederhana menggunakan `RecyclerView`.

7.1 Konsep RecyclerView

Menampilkan data dalam bentuk daftar adalah kebutuhan umum dalam aplikasi mobile. Android menyediakan `RecyclerView` sebagai komponen standar untuk tugas ini. `RecyclerView` adalah versi yang lebih canggih dan fleksibel dari `ListView` yang sudah lama ada.

Keunggulan `RecyclerView`:

- **Reusing Views:** `RecyclerView` mendaur ulang (recycle) item-item yang sudah tidak terlihat di layar untuk menampilkan data baru yang masuk ke layar. Ini secara drastis meningkatkan performa dan mengurangi konsumsi memori, terutama untuk daftar yang sangat panjang.
- **Decoupling:** `RecyclerView` memisahkan tugas menjadi beberapa komponen, membuat kode lebih terorganisir dan mudah dikelola.

Komponen Penting dalam `RecyclerView`:

1. `RecyclerView`: Widget UI itu sendiri yang ditempatkan di layout XML. Tugasnya hanya menampilkan item.
2. `LayoutManager`: Bertanggung jawab untuk mengatur posisi dan ukuran setiap item di layar. Android menyediakan `LinearLayoutManager` (untuk daftar vertikal/horizontal), `GridLayoutManager` (untuk grid), dan `StaggeredGridLayoutManager`.
3. `Adapter`: Jembatan antara sumber data (misalnya, `ArrayList`) dan `RecyclerView`. Tugasnya adalah membuat `ViewHolder` dan mengisi data ke dalam `ViewHolder` berdasarkan posisi item.
4. `ViewHolder`: Objek yang menyimpan referensi ke View untuk satu item di daftar (misalnya, satu `TextView` dan `ImageView` dalam satu baris). Dengan menyimpan referensi ini, kita tidak perlu memanggil `findViewById()` berulang kali, yang meningkatkan efisiensi.

7.2 Implementasi Praktik: Aplikasi Daftar Kontak

Kita akan membuat aplikasi yang menampilkan daftar nama dan nomor telepon kontak.

Langkah 1: Tambahkan Dependensi

Pastikan dependensi `RecyclerView` sudah ada di `build.gradle.kts` (Module :app). Jika tidak, tambahkan:

```
dependencies {  
    // ...  
    implementation("androidx.recyclerview:recyclerview:1.3.2") // Gunakan versi  
    terbaru  
}
```

Langkah 2: Buat Model Data (`Contact.kt`)

Buat file Kotlin baru untuk mendefinisikan struktur data kontak.

```
// ...  
data class Contact(  
    val name: String,  
    val phoneNumber: String  
)
```

Langkah 3: Buat Layout untuk Satu Item (`item_contact.xml`)

Buat file layout baru di `res/layout/item_contact.xml`. Ini adalah tampilan untuk satu baris di daftar kontak.

```
<?xml version="1.0" encoding="utf-8"?>  
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:orientation="vertical"  
    android:padding="16dp">  
  
    <TextView  
        android:id="@+id/tvName"  
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"  
        android:textSize="18sp"  
        android:textStyle="bold"  
        tools:text="John Doe" />  
  
    <TextView  
        android:id="@+id/tvPhoneNumber"  
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"  
        android:textSize="14sp"  
        tools:text="08123456789" />  
  
</LinearLayout>
```

Langkah 4: Buat Adapter dan ViewHolder (`ContactAdapter.kt`)

Buat file Kotlin baru untuk adapter.

```

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import androidx.recyclerview.widget.RecyclerView

class ContactAdapter(private val contactList: List<Contact>) :
    RecyclerView.Adapter<ContactAdapter.ContactViewHolder>() {

    // ViewHolder: Menyimpan referensi ke View dalam satu item layout
    class ContactViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        val tvName: TextView = itemView.findViewById(R.id.tvName)
        val tvPhoneNumber: TextView = itemView.findViewById(R.id.tvPhoneNumber)
    }

    // onCreateViewHolder: Dipanggil saat RecyclerView perlu membuat ViewHolder
    // baru
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
        ContactViewHolder {
        val itemView = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_contact, parent, false)
        return ContactViewHolder(itemView)
    }

    // onBindViewHolder: Dipanggil untuk menghubungkan data dengan ViewHolder pada
    // posisi tertentu
    override fun onBindViewHolder(holder: ContactViewHolder, position: Int) {
        val currentContact = contactList[position]
        holder.tvName.text = currentContact.name
        holder.tvPhoneNumber.text = currentContact.phoneNumber
    }

    // getItemCount: Mengembalikan jumlah total item dalam data
    override fun getItemCount(): Int {
        return contactList.size
    }
}

```

Langkah 5: Tambahkan RecyclerView di Layout Utama (`activity_main.xml`)

```

<!-- activity_main.xml -->
<androidx.constraintlayout.widget.ConstraintLayout ...>
    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/rvContacts"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        tools:listitem="@layout/item_contact" />
</androidx.constraintlayout.widget.ConstraintLayout>

```

Langkah 6: Atur RecyclerView di Activity (MainActivity.kt)

```

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val rvContacts = findViewById<RecyclerView>(R.id.rvContacts)

        // 1. Siapkan data (contoh data statis)
        val contactList = listOf(
            Contact("Alice", "0811111111"),
            Contact("Bob", "0822222222"),
            Contact("Charlie", "0833333333"),
            Contact("Diana", "0844444444"),
            Contact("Eve", "0855555555")
        )

        // 2. Buat instance adapter
        val adapter = ContactAdapter(contactList)

        // 3. Atur layoutManager
    }
}

```

```
rvContacts.layoutManager = LinearLayoutManager(this)

// 4. Atur adapter pada RecyclerView
rvContacts.adapter = adapter
}
}
```

7.3 Studi Kasus Latihan Praktikum (Mini-Proyek)

Studi Kasus: Aplikasi Daftar Tugas (To-Do List)

1. **Buat Model Data:** `Task` dengan properti `title: String` dan `isCompleted: Boolean`.
2. **Buat Layout Item:** `item_task.xml` yang berisi `CheckBox` dan `TextView` untuk judul tugas.
3. **Buat Adapter:** `TaskAdapter` yang menampilkan daftar tugas. Saat `CheckBox` dicentang, ubah gaya teks `TextView` (misalnya, dicoret) dan update status `isCompleted` di dalam model data.
4. **Buat Activity Utama:** Tampilkan `RecyclerView` dengan daftar tugas awal (misalnya, "Belajar Kotlin", "Buat modul", "Olahraga").
5. **Tantangan Tambahan:** Tambahkan `FloatingActionButton` yang ketika diklik, menampilkan `Dialog` atau `Activity` baru untuk menambahkan tugas baru ke dalam daftar. (Ini akan membutuhkan konsep notifikasi perubahan data pada adapter, yang bisa dipelajari lebih lanjut).